Jenkins - CI/CD,
K8s -> container to deploy,
Airflow-> data processing during the execution

Project outlines and Grading Rubrics respectively:

Project 1:

- https://docs.google.com/document/d/1xXoVPJ4gA83WXUU2jA5VYfioZO8Sjn3MmAPOMcPqbY4/edit?usp=sharing
- https://docs.google.com/spreadsheets/d/1m-BaVb4yflejeg5JpE4ijO4fXM2hqifwc0Kv-nHgoJo/edit?usp=sharing
- Keras implementation of Siamese NN:
    - Triplet Loss: https://keras.io/examples/vision/siamese_network/
    - Contrastive Loss: https://keras.io/examples/vision/siamese_contrastive/

Project 2:
https://docs.google.com/document/d/1Ip6S30LZIH5VedhMjzQOecdotTpLNVWDxBBbrLxYo4I/edit?usp=sharing
https://docs.google.com/spreadsheets/d/1RXJQg1aDMH5Mker61ApG9sYhhkTJpodTG5xkiTEdfiE/edit?usp=sharing

GitHub offers 2000 min on public and private repos with your free account. You can use GitHub Actions for the CI/CD pipelines.~ Alejandro

Project 3- Vision Assist:

- Problem Statement: https://docs.google.com/document/d/1sEk7Vph1I9zb-g8PvI6bnc0NnV0JoJFThJ9Un_7cFIE/edit?usp=sharing
- Grading rubrics: https://docs.google.com/spreadsheets/d/1hYQSjnmM_IZixqvhC00vu4VN_bOwfwIs8b9bBbWHGnM/edit?usp=sharing
- How is mAP calculated? (Refer to C23, C24 and C25 in the playlist) https://www.youtube.com/watch?v=QdWidmgLwbw
- 80 classes of COCO dataset (on which YOLO models are trained) https://gist.github.com/AruniRC/7b3dadd004da04c80198557db5da4bda
- How to run predictions on pre-trained YOLO models? (and parameters associated) https://docs.ultralytics.com/modes/predict/
- How to train or fine-tune YOLO models? https://docs.ultralytics.com/modes/train/ https://www.youtube.com/watch?v=ZN3nRZT7b24

- Image explaining precision and recall: https://en.wikipedia.org/wiki/Precision_and_recall#/media/File:Precisionrecall.svg
- Some sample packages for text to speech: pyttsx3 and gTTS

Using Kaggle GPU: https://www.kaggle.com/code/dskagglemt/enabling-and-testing-the-gpu
MlFlow and Airflow: https://www.youtube.com/watch?v=GCY5VX0na6M&t=5s

Project 4:
https://docs.google.com/document/d/1miJfMW4dAczw2nchFwwhpqsXZ8koJogr/edit?usp=sharing&ouid=111235319467332638658&rtpof=true&sd=true
https://docs.google.com/spreadsheets/d/1C8VsCu-hEww6CJt_By964I-KSXTGfhcz/edit?usp=sharing&ouid=111235319467332638658&rtpof=true&sd=true

Project 5- Resume Coach:
- Problem Statement: https://docs.google.com/document/d/1QJjgJLLsU3SYxsETjMXltuJecDwWIUt-/edit?usp=sharing&ouid=111235319467332638658&rtpof=true&sd=true
- Grading rubrics: https://docs.google.com/spreadsheets/d/1xL1YcLBEthXTTZ_-bFd80sr45mgObUuGtHuXmuLMMgk/edit?usp=sharing
- Getting started with AWS Sagemaker: https://docs.google.com/document/d/1-TwFJfoQCdOcf9ByLWLm7BR3U_T5UNk5/edit?usp=sharing&ouid=111235319467332638658&rtpof=true&sd=true
- Getting started with Replicate: https://docs.google.com/document/d/1iSp9r_02K4JzkRNOY4bqzdb-x40nGpn09NdQkmV0haQ/edit?usp=sharing
- Action Plan: https://docs.google.com/document/d/15K0oiElhl7deAlCE1cS6vcp1yG6TzruD/edit?usp=sharing&ouid=111235319467332638658&rtpof=true&sd=true

Project 6- ShopTalk:
- Problem Statement: https://docs.google.com/document/d/1HcTq3ARUTYYH7VOTJKtgDALZ4dOzhpoCUasjHc34aNw/edit?usp=sharing
- Grading rubrics: https://docs.google.com/spreadsheets/d/19ZtD1E07XihizJM5mjKnlcIvmOMOnr9eeX3EcQEpaag/edit?usp=sharing

Links shared during the session:
Resume Coach:
- https://www.jobscan.co/

- https://replicate.com/
- Sample datasets:
  https://www.kaggle.com/datasets/promptcloud/indeed-job-posting-dataset
  https://www.kaggle.com/datasets/mdwaquarazam/data-science-data-analyst-and-ml-job-indeed
- https://towardsdatascience.com/saving-and-loading-pytorch-models-in-kaggle-3dadc0af1bd9
- https://www.kaggle.com/code/dskagglemt/enabling-and-testing-the-gpu
- Spot Instances vs On Demand Instances - AWS:
  https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/using-spot-instances.html
- Generic RAG Architecture:
  https://github.com/just-joseph/ApologyGenerator_Assignment/blob/main/Architecture.png
  A simple sample application:
  https://github.com/just-joseph/ApologyGenerator_Assignment/blob/main/Apologygenerator_RAG_JustinJoseph.py
- Ollama:
  https://github.com/ollama/ollama
- Open WebUI:
  https://github.com/open-webui/open-webui
- For Langchain beginners,
  https://python.langchain.com/docs/tutorials/
- Some LLM evaluation frameworks (when you don't have ground truth/ labels) (LLM-as-an-evaluator):
  - Openai evals
  - Ragas:
    https://medium.com/data-science/evaluating-rag-applications-with-ragas-81d67b0ee31a
  - Promptfoo
  - Trulens
- You can look into metrics like relevance, coverage, faithfulness, fluency, specificity, helpfulness etc. which can be rated by the evaluator LLM.

Shoptalk:

- https://amazon-berkeley-objects.s3.amazonaws.com/index.html
- How to use 3D models?
  https://www.aboutwayfair.com/tech-blog/machine-learning-to-aid-3d-modeling-at-wayfair
- A package that helps deploy Jupyter Notebooks
  https://papermill.readthedocs.io/en/latest/
- Image captioning models that you can check out:
  - https://openai.com/index/clip/
  - https://clarifai.com/salesforce/blip
  - https://imagebind.metademolab.com/ (Imagebind is multimodal in nature)
  - https://www.anthropic.com/claude/sonnet (multimodal) (commercial)

    - You can also checkout open source models at https://huggingface.co/ (For eg, https://huggingface.co/Salesforce/blip-image-captioning-large)
- Task Adaptation without Fine Tuning paper: https://arxiv.org/pdf/2310.14034
- TopK Algorithms paper: https://www.cs.ubc.ca/~laks/topK-techReport.pdf
- If images need to be directly converted to embeddings, try using CLIP/ Dino/ ViT.

BinSense AI:

- Some popular labeling tools for images (object detection):
  Labelimg
  Labelme
  CVAT
  LabelStudio
- A popular data augmentation package for Python:
  https://github.com/albumentations-team/albumentations
- Some methods to improve annotation speed:
  Try SAM from meta: https://segment-anything.com/demo#
  Or try semi-supervised approaches

RAG/ LLMs/ Agents:

- LoRA, QLoRA intuition:
  https://www.youtube.com/watch?v=6S59Y0ckTm4
  https://www.youtube.com/watch?v=l5a_uKnbEr4
  Alternative source:
  https://www.youtube.com/watch?v=t1caDsMzWBk
  Simple explanation:
  https://www.youtube.com/watch?v=lixMONUAjfs
- Fine-tune LLAMA2 using LoRA and QLoRA approach:
  https://www.youtube.com/watch?v=Vg3dS-NLUT4
-

Criteria	**LORA**	**QLORA**
Definition	Fine tune by learning low rank updates	LORA + quantization of weights/biases (parameters)
Efficiency	Reduces number of fine-tuned parameters	Further reduces memory & computation requirements through quantization
Memory usage	Moderate	Very low
Performance hit	Minimal loss in performance	Slightly lower accuracy. If quantization is aggressive, accuracy hit will be higher
Hardware requirement	Resource limited setting	Highly resource constrained setting

- https://levelup.gitconnected.com/mastering-langchain-with-examples-the-future-of-ai-development-180bea2a6d27